

**МОСКОВСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИМЕНИ М.В. ЛОМОНОСОВА**

ФИЛИАЛ В ГОРОДЕ ДУШАНБЕ

**Естественно-научный факультет
направление подготовки 01.03.02 Прикладная математика и
информатика**



ОТЧЁТ О ПРОХОЖДЕНИИ ПРЕДДИПЛОМНОЙ ПРАКТИКИ

с 4 мая по 16 мая 2026 года

Выполнил:

студент 4 курса естественно-научного факультета направления подготовки 01.03.02 Прикладная математика и информатика
Хайбулов Дамир Ахлидинович

Руководитель практики от Организации:

Руководитель отдела разработки
Изатшоев О. А.

Руководитель практики от Филиала:

к.ф.-м.н., доцент, зав. кафедрой математики и естественных наук
Одинабеков Джасур Музофирович

Душанбе – 2026

СОДЕРЖАНИЕ

Введение	5
ГЛАВА 1 Цели, задачи и содержание преддипломной практики . .	8
ГЛАВА 2 Теоретические основы решения уравнения переноса . . .	11
2.1 Математическая постановка задачи	11
2.2 Метод характеристик	12
2.3 Типовые случаи коэффициента переноса	13
2.4 Разностные схемы и дискретизация	13
2.5 Неявные методы и решение систем линейных уравнений . . .	15
2.6 Устойчивость, число Куранта и нормы ошибок	16
ГЛАВА 3 Требования к программной системе	18
3.1 Функциональные требования	18
3.2 Нефункциональные требования	19
3.3 Сценарии использования	21
ГЛАВА 4 Проектирование архитектуры приложения	22
4.1 Выбор технологического стека	22
4.2 Логическая структура системы	22
4.3 Организация набора схем	24
4.4 Поддержка пользовательских схем	25
4.5 Интерфейс пользователя	27
ГЛАВА 5 Программная реализация системы	28
5.1 Построение вычислительного контекста	28
5.2 Реализация явных схем	28

5.3	Реализация неявных схем	30
5.4	Расчёт ошибок и анализ сходимости	31
5.5	Визуализация и экспорт	33
ГЛАВА 6	Результаты вычислительных экспериментов	35
6.1	Организация экспериментов	35
6.2	Сравнение явных схем	35
6.3	Роль ограничителей и схем TVD/MUSCL	37
6.4	Сравнение неявных схем	37
6.5	Пользовательские схемы как инструмент исследования	38
6.6	Анализ ошибок и таблицы норм	38
6.7	Анализ сходимости	39
6.8	Интерпретация результатов с точки зрения учебного процесса	39
ГЛАВА 7	Тестирование, отладка и оценка качества	41
7.1	Стратегия тестирования	41
7.2	Тестовые сценарии	41
7.3	Анализ производительности	43
7.4	Ограничения текущей версии	43
ГЛАВА 8	Практические результаты и сформированные компетенции	44
ГЛАВА 9	Перспективы развития проекта	46
	Заключение	47
	Список использованных источников	48
ГЛАВА А	Руководство пользователя	50
A.1	Запуск программы	50
A.2	Базовый сценарий работы	50
A.3	Добавление пользовательской схемы	51

A.4	Интерпретация результатов	51
ГЛАВА В	Каталог реализованных схем	53
В.1	Встроенные явные схемы	53
В.2	Встроенные неявные схемы	54
ГЛАВА С	Фрагменты конфигурации и кода	56

ВВЕДЕНИЕ

Современная инженерная и научная деятельность всё в большей степени опирается на вычислительный эксперимент. Для значительного класса задач математической физики аналитическое решение либо отсутствует, либо существует только для ограниченного набора частных случаев. Поэтому качество подготовки специалиста в области прикладной математики и программной инженерии во многом определяется не только знанием теории, но и умением разрабатывать инструменты для численного исследования моделей, анализировать устойчивость алгоритмов и интерпретировать полученные результаты.

Одним из базовых объектов изучения в вычислительной математике является уравнение переноса. Оно возникает при описании конвективных процессов, движения примесей, распространения сигналов и иных явлений, в которых некоторое поле перемещается вдоль характеристик без внутреннего источника. Несмотря на простую формальную запись, эта модель демонстрирует широкий спектр методических вопросов: влияние выбора схемы на диссипацию и дисперсию, зависимость результата от числа Куранта, различие между явными и неявными подходами, роль искусственной вязкости, чувствительность к разрывным и гладким начальным данным.

Для эффективного изучения указанных аспектов недостаточно иметь набор отдельных скриптов. Требуется программное средство, которое объединяет построение аналитического решения, набор типовых разностных схем, расчёт ошибок и удобную визуализацию. Особенно важна возможность быстро переключаться между режимами коэффициента переноса a , изменять сеточные параметры, сравнивать несколько методов на одном графике и отслеживать эволюцию решения во времени. Такие функции превращают программу в лабораторию численного анализа, пригодную и для индивидуаль-

ного исследования, и для учебного процесса.

Целью преддипломной практики являлась разработка настольного приложения для исследования численных схем решения одномерного уравнения переноса с акцентом на воспроизводимость вычислительных экспериментов, удобство интерфейса и расширяемость архитектуры. Для достижения цели были поставлены следующие задачи:

1. изучить математическую постановку задачи и выделить те случаи, для которых возможно построение аналитического эталона;
2. определить перечень численных схем, которые должны войти в базовую поставку приложения;
3. сформулировать функциональные и нефункциональные требования к интерфейсу и вычислительному модулю;
4. реализовать механизм параметризации коэффициента переноса и начального условия;
5. реализовать набор методов анализа качества решения: нормы ошибок, анализ сходимости, расчёт CFL-числа, экспорт графических результатов;
6. провести вычислительные эксперименты и подготовить документированное описание полученного результата.

Объектом исследования является процесс численного решения одномерного уравнения переноса, а предметом исследования выступают алгоритмы и программные средства, обеспечивающие его интерактивное изучение в настольном приложении. Практическая значимость работы состоит в создании инструмента, который может быть использован как элемент учебно-методического обеспечения, как основа для выпускной квалификационной

работы и как стартовая платформа для дальнейшего расширения функциональности в сторону более сложных моделей.

ГЛАВА 1 ЦЕЛИ, ЗАДАЧИ И СОДЕРЖАНИЕ ПРЕДДИПЛОМНОЙ ПРАКТИКИ

Преддипломная практика является завершающим этапом подготовки выпускника, на котором проверяется способность применять накопленные знания в рамках цельного инженерного проекта. В отличие от учебной или производственной практики, на данном этапе акцент смещается на самостоятельную постановку технических решений, обоснование выбранных архитектурных и алгоритмических подходов, а также на формирование результатов, которые могут быть непосредственно использованы в дипломном проектировании.

В рамках рассматриваемой работы преддипломная практика была организована как последовательность взаимосвязанных этапов. На первом этапе была выполнена аналитическая проработка предметной области. Необходимо было сопоставить математическое содержание уравнения переноса с требованиями к программному продукту. Выяснилось, что для учебно – исследовательского приложения принципиально важно поддержать не один, а несколько режимов задания коэффициента a : постоянный коэффициент, зависимость от пространственной переменной, зависимость от времени и смешанную зависимость от x и t . Это позволяет охватить разные классы задач и демонстрирует особенности поведения схем на существенно различных постановках.

На втором этапе была сформирована концепция программного продукта. Приложение должно было выполнять сразу несколько ролей: служить калькулятором аналитического решения, инструментом сравнения численных методов, средством визуального анализа и платформой для пользовательских экспериментов. Следовательно, система должна быть интерактивной, расширяемой и устойчивой к некорректному вводу. Из этого напрямую следовали

требования к выбору технологий: язык разработки должен иметь сильную экосистему для численных расчётов и интерфейсов, а графическая библиотека обязана обеспечивать построение интерактивных визуализаций.

На третьем этапе была выполнена детальная проработка алгоритмов. Помимо реализации классических явных схем против потока, Лакса – Фридрихса, Лакса – Вендроффа и Мак – Кормака, в проект были включены TVD- и MUSCL-подходы, варианты с искусственной вязкостью, а также ряд неявных схем, включая центральные, апвиндные и регуляризованные методы. Отдельной задачей стало внедрение настраиваемых пользовательских схем, задаваемых формульно. Это требовало разработки формата хранения, валидации, интерпретации выражений и согласования их с общим вычислительным контуром.

На четвёртом этапе выполнялась разработка интерфейса. Необходимо было организовать экран так, чтобы пользователь мог за один сеанс работы изменить тип коэффициента переноса, диапазоны сетки, число узлов, набор сравниваемых схем, режим отображения аналитического решения и параметры визуального окна. Параллельно была предусмотрена область текстового вывода, в которую помещаются отчёты о расчёте, ошибки, предупреждения и краткие итоги анализа.

На заключительных этапах практика была направлена на отладку, тестирование и подготовку документации. Были проверены типовые сценарии, оценены узкие места, внесены коррективы в обработку ошибок и механизм экспорта анимаций. Итогом стала программная система, которая демонстрирует полноценный цикл разработки: от математической постановки и алгоритмизации до пользовательского интерфейса и описания результатов.

Содержательно практика позволила закрепить и объединить знания по следующим направлениям:

- математическое моделирование и постановка задачи переноса;

- численные методы решения задач математической физики;
- программирование на Python и работа с научными библиотеками;
- проектирование графических пользовательских интерфейсов;
- сериализация пользовательских данных и обеспечение расширяемости системы;
- визуальный анализ данных и подготовка отчётной документации.

Таким образом, преддипломная практика не свелась к написанию отдельного скрипта, а представляла собой инженерную работу по созданию законченного программного продукта, ориентированного на решение учебно-исследовательских задач в области численного анализа.

ГЛАВА 2 ТЕОРЕТИЧЕСКИЕ ОСНОВЫ РЕШЕНИЯ УРАВНЕНИЯ ПЕРЕНОСА

2.1 Математическая постановка задачи

В базовом виде одномерное уравнение переноса можно записать как

$$\frac{\partial u}{\partial t} + a(x, t) \frac{\partial u}{\partial x} = 0, \quad (2.1)$$

где $u(x, t)$ — искомая функция, а $a(x, t)$ — скорость переноса, которая в зависимости от постановки может быть постоянной или переменной. Начальное условие задаётся функцией

$$u(x, 0) = \varphi_0(x). \quad (2.2)$$

Смысл уравнения состоит в том, что профиль u переносится вдоль характеристик, причём форма решения в простейших случаях сохраняется. Именно это делает задачу удобной для учебного анализа: с одной стороны, модель достаточно проста для построения аналитических эталонов, с другой — она чувствительна к дискретизации и позволяет наглядно увидеть особенности различных разностных схем.

Если коэффициент переноса постоянен, то решение имеет вид

$$u(x, t) = \varphi_0(x - at). \quad (2.3)$$

То есть начальный профиль просто смещается без изменения формы. Однако при численном решении реальное поведение зависит от выбранной схемы: одни методы искусственно сглаживают профиль, другие создают паразитические колебания, третьи дают приемлемый баланс между устойчиво-

стью и точностью. Поэтому уравнение переноса является классическим тестом для сравнения численных методов.

2.2 Метод характеристик

Для аналитического исследования задачи переноса ключевым инструментом является метод характеристик. Идея метода состоит в поиске кривых $x(t)$ в плоскости переменных, вдоль которых значение решения сохраняется или изменяется по упрощённому закону. Для рассматриваемой модели характеристические уравнения принимают вид

$$\frac{dx}{dt} = a(x, t), \quad \frac{du}{dt} = 0. \quad (2.4)$$

Второе соотношение означает, что вдоль характеристики величина u постоянна. Следовательно, если известна точка пересечения характеристики с начальной линией $t = 0$, то значение решения в любой последующей точке определяется начальным условием. Отсюда возникает общая идея аналитического решения: найти отображение, которое каждой точке (x, t) ставит в соответствие начальную координату ξ , и затем вычислить

$$u(x, t) = \varphi_0(\xi(x, t)). \quad (2.5)$$

В разработанном приложении эта идея использована как для прямого вычисления аналитического решения, так и для визуализации характеристик. Для случая переменного коэффициента транспортировки аналитические выражения определены для нескольких типовых зависимостей: $a = a(x)$, $a = a(t)$ и ряда каталогизированных случаев $a = a(x, t)$. Таким образом, интерфейс не ограничивается только постоянной скоростью переноса и позволяет демонстрировать более содержательные сценарии.

2.3 Типовые случаи коэффициента переноса

Для зависимости $a = a(x)$ характеристическое описание можно выразить через интеграл

$$F(x) = \int \frac{dx}{a(x)}. \quad (2.6)$$

Тогда координата начальной точки восстанавливается по соотношению $F(\xi) = F(x) - t$. В программной реализации эта идея реализуется через численное интегрирование. Это позволяет обойтись без символьного вывода замкнутой формулы, сохраняя при этом связь с характеристическим подходом.

Для зависимости $a = a(t)$ характерен другой тип преобразования. Пусть

$$S(t) = \int_0^t a(\tau) d\tau. \quad (2.7)$$

Тогда аналитическое решение запишется как

$$u(x, t) = \varphi_0(x - S(t)). \quad (2.8)$$

Наконец, для каталога функций $a(x, t)$ в программе предусмотрены специально подготовленные случаи, например $a = x - t$, $a = x + t$, $a = xt$, $a = x \exp(-t)$ и аффинные комбинации вида $a = \alpha x + \beta t + d$. Для них построены формулы, позволяющие получать эталонное решение и сравнивать его с численными результатами.

Поддержка этих вариантов имеет важное методическое значение. Пользователь может увидеть, что даже при сохранении общей структуры уравнения свойства решения и требования к сетке меняются. Следовательно, приложение позволяет изучать не только отдельный метод, но и его поведение на семействах задач.

2.4 Разностные схемы и дискретизация

Пусть пространство и время дискретизированы равномерными сетками

$$x_i = x_{\min} + ih, \quad t^n = t_{\min} + n\tau, \quad (2.9)$$

где h — пространственный шаг, τ — временной шаг. Тогда численное решение обозначается через $u_i^n \approx u(x_i, t^n)$. Для уравнения переноса естественным является введение безразмерного параметра

$$\lambda_i^n = \frac{a_i^n \tau}{h}, \quad (2.10)$$

который характеризует относительное смещение информации за один временной шаг.

В приложении реализованы следующие основные группы схем:

- явные апвиндные и центральные схемы;
- явные схемы второго порядка, включая Лакса – Вендроффа и Мак – Кормака;
- TVD- и MUSCL-подходы с ограничителями потоков;
- схемы с искусственной линейной и нелинейной вязкостью;
- неявные трёхдиагональные и пентадиагональные схемы;
- пользовательские схемы, заданные формулами для обновления узлов или коэффициентов матрицы.

Явная схема против потока для $a > 0$ имеет вид

$$u_i^{n+1} = u_i^n - \lambda_i^n (u_i^n - u_{i-1}^n). \quad (2.11)$$

Она проста и надёжна, но обладает численной вязкостью, сглаживающей решение. Схема Лакса–Фридрихса заменяет центральную производную на комбинацию усреднения и разности:

$$u_i^{n+1} = \frac{u_{i+1}^n + u_{i-1}^n}{2} - \frac{\lambda_i^n}{2}(u_{i+1}^n - u_{i-1}^n). \quad (2.12)$$

Схема Лакса–Вендроффа повышает порядок аппроксимации за счёт добавления второго разностного члена:

$$u_i^{n+1} = u_i^n - \frac{\lambda_i^n}{2}(u_{i+1}^n - u_{i-1}^n) + \frac{(\lambda_i^n)^2}{2}(u_{i+1}^n - 2u_i^n + u_{i-1}^n). \quad (2.13)$$

С ростом порядка точности растёт и чувствительность к разрывам и крутым фронтам. Именно поэтому в реальных вычислительных экспериментах востребованы TVD- и MUSCL-подходы, которые подавляют нефизические осцилляции с помощью ограничителей. В проекте реализованы варианты с ограничителями minmod и MC, что даёт возможность исследовать компромисс между монотонностью и точностью.

2.5 Неявные методы и решение систем линейных уравнений

Неявные схемы требуют решения линейной системы на каждом временном шаге. Их основное преимущество состоит в большей устойчивости, особенно при сравнительно крупных временных шагах. Цена за это — увеличение вычислительных затрат и необходимость организовать надёжный решатель для ленточных матриц.

В разработанном приложении применяются два подхода. Первый связан с использованием трёхдиагональной структуры, для которой реализован метод прогонки. Второй подход опирается на матричное описание пользовательской схемы и использует решатель для ленточных систем из библиотеки SciPy. Такое разделение позволяет с одной стороны эффективно вычислять стандартные схемы, а с другой — дать пользователю гибкий механизм задания собственной матрицы.

С инженерной точки зрения это важный архитектурный выбор. Если все схемы реализовать в одном стиле, код будет менее гибким. Если же допустить несколько форматов схем и унифицировать их через общую функцию разрешения имени метода, можно сохранить расширяемость без усложнения интерфейса пользователя. Именно такой подход реализован в проекте.

2.6 Устойчивость, число Куранта и нормы ошибок

Для гиперболических задач важнейшей характеристикой является число Куранта–Фридрихса–Леви:

$$\text{CFL} = \max_{x,t} |a(x, t)| \frac{\tau}{h}. \quad (2.14)$$

Если значение CFL слишком велико, многие явные схемы становятся неустойчивыми. В приложении предусмотрен специальный инструмент, вычисляющий число CFL по заданной сетке и параметрам коэффициента. Пользователь получает не только численное значение, но и краткую интерпретацию, позволяющую быстро определить потенциальный риск потери устойчивости.

Для оценки точности численного решения реализован расчёт норм ошибок относительно аналитического эталона. Используются нормы

$$\|e\|_{L_1} = h \sum_i |u_i - \tilde{u}_i|, \quad (2.15)$$

$$\|e\|_{L_2} = \left(h \sum_i (u_i - \tilde{u}_i)^2 \right)^{1/2}, \quad (2.16)$$

$$\|e\|_{L_\infty} = \max_i |u_i - \tilde{u}_i|, \quad (2.17)$$

где $\tilde{u}_i$ обозначает аналитическое решение. Одновременный анализ несколь-

ких норм даёт более полную картину. Например, схема может демонстрировать умеренную ошибку в L_2 , но иметь заметные локальные выбросы в L_∞ . В контексте учебного исследования это особенно полезно, поскольку позволяет обсуждать не только среднюю точность, но и поведение схемы в критических точках.

ГЛАВА 3 ТРЕБОВАНИЯ К ПРОГРАММНОЙ СИСТЕМЕ

3.1 Функциональные требования

После анализа предметной области были сформулированы требования к разрабатываемому приложению. В первую очередь система должна обеспечивать полный цикл исследования: от задания параметров модели до получения визуальных и численных выводов. Это означает, что графический интерфейс не может ограничиваться одной кнопкой расчёта; он обязан поддерживать интерактивный сценарий работы с множеством параметров и режимов отображения.

К основным функциональным требованиям относятся:

№	Требование	Содержание
1	Задание модели	Выбор режима коэффициента переноса: постоянный, зависящий от x , зависящий от t , либо выбираемый из каталога $a(x, t)$.
2	Задание начального условия	Ввод аналитического выражения $\varphi_0(x)$ с последующим преобразованием к вычислимой функции.
3	Настройка сетки	Ввод границ области по пространству и времени, а также количества узлов по каждой координате.
4	Выбор схем	Отметка одной или нескольких встроенных или пользовательских схем для одновременного сравнения.

5	Визуализация	Построение анимации, срезов, контурных и трёхмерных графиков.
6	Анализ ошибок	Расчёт норм ошибок, формирование таблиц итоговых значений, визуализация динамики норм.
7	Анализ сходимости	Построение графика зависимости ошибки от шага сетки для выбранной схемы.
8	Расчёт CFL	Определение числа Куранта и выдача краткой оценки устойчивости.
9	Работа с пользовательскими схемами	Добавление, удаление, сохранение и повторная загрузка собственных схем пользователя.
10	Экспорт результатов	Сохранение анимации в форматах GIF и MP4 с индикацией прогресса.

Следует отметить, что требования к пользовательским схемам были сформулированы отдельно. Пользователь должен иметь возможность задать либо явную формулу перехода на новый слой, либо неявную схему через описание правой части и коэффициентов матрицы. Такое решение существенно расширяет образовательный потенциал системы: приложение превращается из статичного демонстратора в площадку для постановки собственных вычислительных экспериментов.

3.2 Нефункциональные требования

Нефункциональные требования в рассматриваемом проекте не менее важны, чем функциональные. Система ориентирована на учебную и исследовательскую работу, поэтому ошибки ввода, зависания интерфейса или невоз-

возможность повторить эксперимент снижают ценность продукта даже при корректно реализованных формулах. По этой причине были выделены следующие нефункциональные требования:

- **наглядность:** все ключевые результаты должны быть доступны в визуальной форме;
- **отказоустойчивость:** программа обязана корректно сообщать пользователю о некорректном вводе, а не завершаться аварийно;
- **расширяемость:** добавление новой схемы не должно требовать перестройки архитектуры;
- **повторяемость экспериментов:** пользовательские схемы и настройки должны сохраняться между запусками;
- **приемлемая производительность:** расчёты на сетках средней плотности должны выполняться без ощутимых задержек;
- **кроссплатформенность:** приложение должно запускаться в типичной настольной среде без жёсткой привязки к конкретной операционной системе.

Отдельно стоит подчеркнуть требование к объяснимости результатов. В системе присутствует текстовая область вывода, где отображаются краткие отчёты, сообщения об ошибках, итоги расчётов и список пропущенных схем. Это важная деталь пользовательского опыта, поскольку при одновременном сравнении нескольких методов пользователь должен понимать, какие вычисления завершились успешно, а какие были пропущены из-за несоответствия условий применимости.

3.3 Сценарии использования

Для оценки полноты требований были выделены типовые сценарии использования:

1. студент выбирает начальное условие $\varphi_0(x) = \exp(-x^2)$, включает несколько схем и сравнивает их на одном графике;
2. преподаватель демонстрирует влияние CFL-числа на устойчивость явной схемы, последовательно изменяя сеточные параметры;
3. исследователь добавляет собственную схему через формулу, сохраняет её в конфигурацию и сопоставляет с базовыми методами;
4. пользователь экспортирует анимацию эволюции решения для включения в презентацию или отчёт;
5. разработчик проверяет порядок сходимости выбранной схемы, последовательно уточняя сетку и анализируя логарифмический график ошибки.

Эти сценарии напрямую определили состав интерфейса и логику взаимодействия между вычислительным ядром и средствами визуализации.

ГЛАВА 4 ПРОЕКТИРОВАНИЕ АРХИТЕКТУРЫ ПРИЛОЖЕНИЯ

4.1 Выбор технологического стека

В качестве языка реализации выбран Python. Такое решение обусловлено несколькими факторами. Во-первых, Python является фактическим стандартом для быстрого прототипирования вычислительных приложений и учебных научных систем. Во-вторых, экосистема языка включает библиотеки NumPy, SciPy и SymPy, которые покрывают основные вычислительные потребности проекта: векторные операции, численное интегрирование, решение систем линейных уравнений и интерпретацию символьных выражений. В-третьих, наличие PyQt5 и Plotly позволяет совместить настольный интерфейс с современными интерактивными графиками.

Для графического интерфейса использована библиотека PyQt5. Она предоставляет зрелую модель виджетов, механизмы сигналов и слотов, а также удобные средства компоновки экранов. Важным аргументом в её пользу стало наличие диалоговых окон, списков, таблиц, полос прогресса и потоков, которые понадобились для реализации полноценного пользовательского интерфейса.

Для визуализации результатов применён Plotly. По сравнению со статическими средствами отображения он предоставляет интерактивные анимации, элементы управления воспроизведением, ползунки времени, масштабирование и экспорт в веб-представление. В контексте учебной задачи это особенно полезно: пользователь может не только посмотреть итоговый график, но и динамически исследовать эволюцию решения.

4.2 Логическая структура системы

Архитектура приложения построена вокруг нескольких логических подсистем:

1. **подсистема построения контекста задачи;**
2. **подсистема вычислительных схем;**
3. **подсистема анализа ошибок и сходимости;**
4. **подсистема визуализации;**
5. **подсистема работы с пользовательскими схемами;**
6. **подсистема графического интерфейса и диалогов.**

Ключевым архитектурным объектом является контекст задачи. В нём хранятся пространственная и временная сетки, функция коэффициента переноса, функция аналитического решения, а также служебные признаки, например факт постоянства коэффициента и его численное значение. Такая инкапсуляция позволяет передавать схемам единый объект с согласованным набором данных. Следовательно, каждая схема получает одинаковый интерфейс входа и не зависит от деталей пользовательского интерфейса.

С практической точки зрения это снижает связанность модулей. Вычислительный код не знает о наличии полей ввода, кнопок и выпадающих списков; он получает только готовый контекст. Аналогично, интерфейс не содержит в себе математической логики схем, а лишь инициирует расчёт, собирает параметры и выводит результаты. Подобное разделение соответствует принципам проектирования расширяемых систем и облегчает дальнейшее развитие проекта.

4.3 Организация набора схем

Встроенные схемы сгруппированы в два словаря: явные и неявные (рисунок (4.1)).

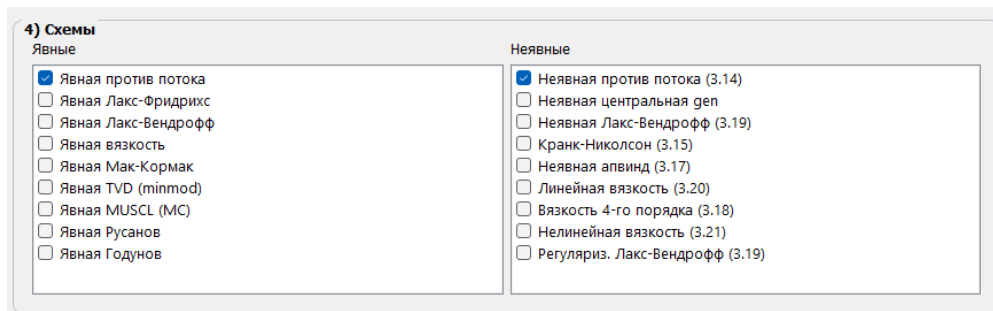


Рисунок 4.1. Группировка явных и неявных схем

Такое решение даёт несколько преимуществ. Во-первых, интерфейс может автоматически сформировать список доступных методов без ручного дублирования имён. Во-вторых, добавление новой схемы сводится к включению функции в соответствующий словарь. В-третьих, пользователь получает естественное разделение методов по классу реализации.

Для справочного режима создан отдельный каталог текстовых формул, рисунок (4.2).

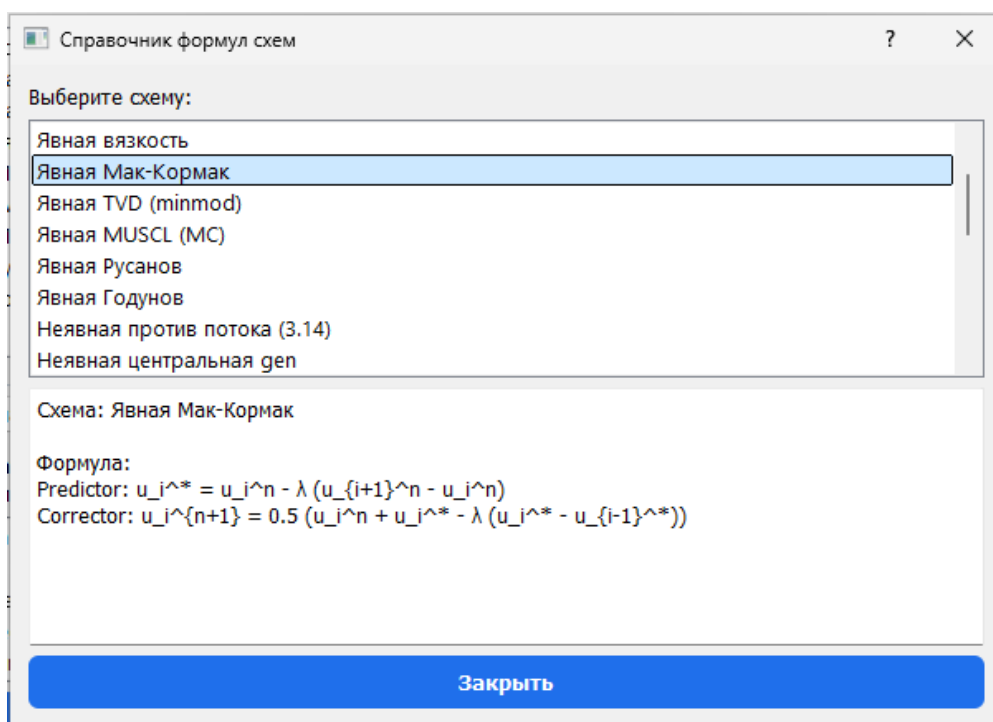


Рисунок 4.2. Справочник схем

Он содержит компактные описания схем в математическом виде и используется в диалоговом окне справочника. Это важный элемент методической поддержки: пользователь не просто выбирает неизвестное имя алгоритма, а может быстро ознакомиться с формулой и сопоставить её с наблюдаемым поведением на графике.

4.4 Поддержка пользовательских схем

Одной из наиболее значимых архитектурных особенностей проекта является механизм пользовательских схем (рисунок (4.3)).

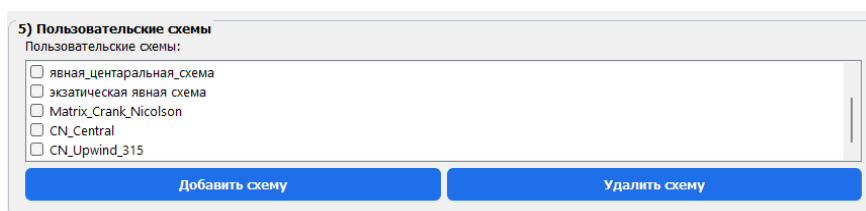


Рисунок 4.3. Интерфейс для пользовательских схем

Система поддерживает три типа пользовательского описания, показанных на рисунке (4.4):

- явная схема, задаваемая формулой для вычисления нового значения узла;
- неявная схема в режиме шаблонов, где пользователь выбирает тип матричной структуры и задаёт правую часть;
- матричная неявная схема, для которой пользователь непосредственно задаёт диагонали ленточной матрицы и выражение для правой части.

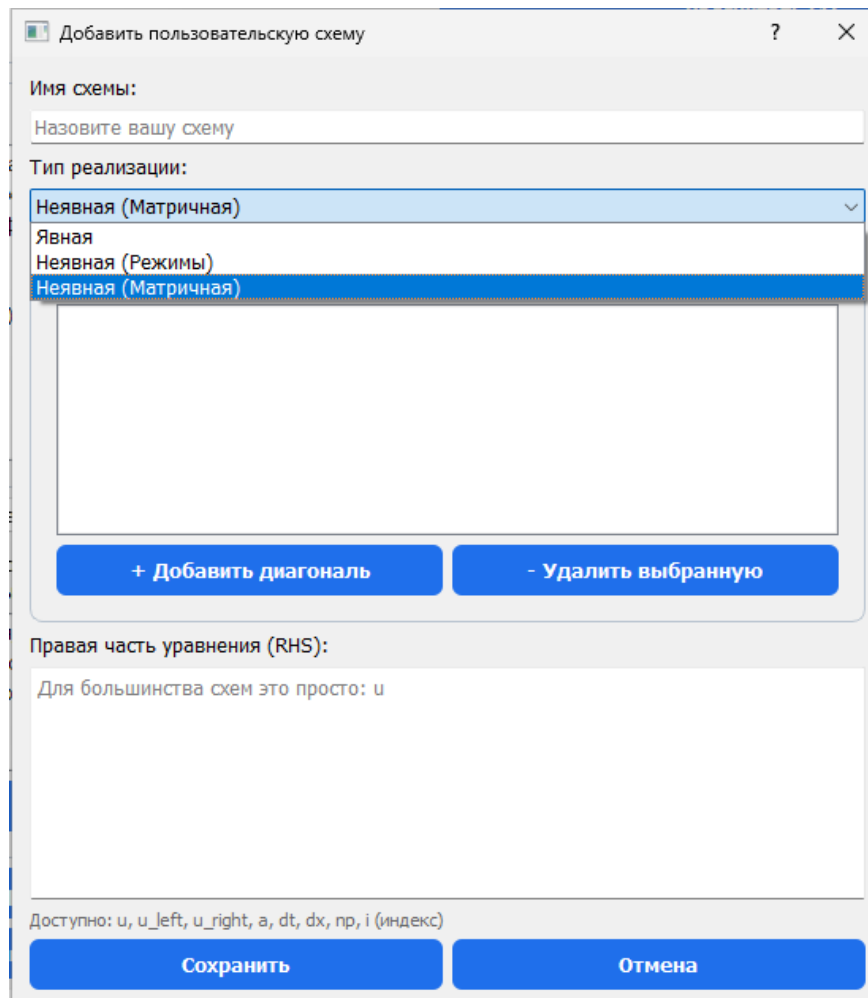


Рисунок 4.4. Режимы ввода пользовательских схем

Данные пользовательских схем хранятся в JSON-файле. Это обеспечивает переносимость и повторяемость экспериментов. При запуске приложения схемы автоматически загружаются, а при изменении списка сохраняются обратно. В результате пользователь один раз настраивает нужный набор методов и далее может использовать его без повторного ввода.

Такой подход демонстрирует важный принцип программной инженерии: расширяемость достигается не через редактирование исходного кода, а через выделение устойчивого формата конфигурации. В данном проекте пользователь фактически получает мини-язык задания схем, встроенный в настольное приложение.

4.5 Интерфейс пользователя

Экран приложения разделён на логические блоки: задание коэффициента переноса и начального условия, настройка сетки, параметры окна графика, расчёт CFL, выбор схем, список пользовательских схем, кнопки анализа и область текстового вывода. Такое разбиение позволяет последовательно пройти путь от постановки задачи к анализу результата. Интерфейс показан на рисунке (4.5)

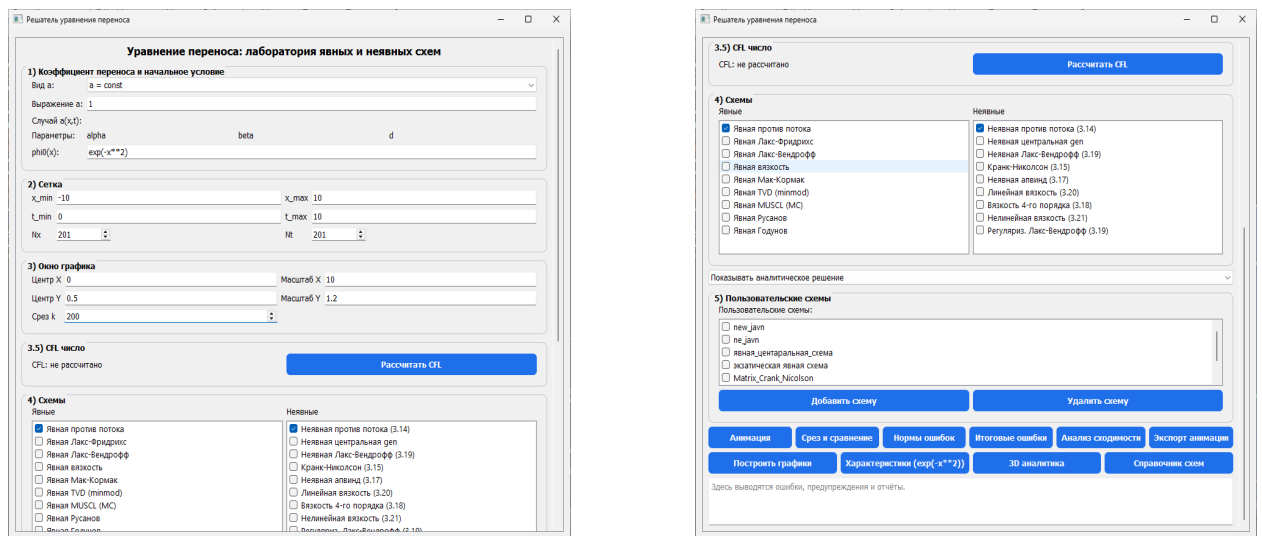


Рисунок 4.5. Интерфейс программы представленный в двух рисунках

Интерфейс ориентирован на компактность без перегрузки деталями. Большинство действий выполняется с главного окна, а дополнительные операции вынесены в отдельные диалоговые формы. Например, добавление пользовательской схемы реализовано в отдельном диалоге, а таблица итоговых норм ошибок отображается в модальном окне. Это позволяет сохранить основной экран достаточно чистым и не перегружать пользователя редкими настройками.

ГЛАВА 5 ПРОГРАММНАЯ РЕАЛИЗАЦИЯ СИСТЕМЫ

5.1 Построение вычислительного контекста

Создание контекста задачи начинается с чтения пользовательских параметров: границ по пространству и времени, числа узлов, выражения для начального условия и вида коэффициента переноса. Далее формируются равномерные сетки и строятся функции, описывающие коэффициент $a(x, t)$ и аналитическое решение $u(x, t)$. Такой подход позволяет единообразно работать как с постоянным коэффициентом, так и с более сложными зависимостями.

При интерпретации пользовательских выражений используется библиотека SymPy. Она преобразует строковую формулу в символьное выражение, после чего создаётся численная функция с использованием NumPy-модулей. Достоинство этого решения заключается в гибкости: пользователь может вводить выражения наподобие $\exp(-x^2)$, $\sin(x)$ или комбинации с функцией `heaviside`. Недостатком является необходимость тщательно обрабатывать ошибки разбора, что и реализовано в приложении через исключения и сообщения в интерфейсе.

Для случая $a = a(x)$ программа дополнительно проверяет конечность значений, отсутствие нулей на сетке и сохранение знака коэффициента. Эти проверки принципиально важны, поскольку аналитическое представление через характеристический интеграл и обратную интерполяцию требует монотонности соответствующего преобразования. Таким образом, система предотвращает вычислительные сценарии, которые привели бы либо к некорректным результатам, либо к неочевидным сбоям.

5.2 Реализация явных схем

Каждая явная схема оформлена как отдельная функция, принимающая контекст и возвращающая двумерный массив решения. Внутри временного цикла производится вычисление коэффициента Куранта и обновление внутренних узлов. Граничные значения восстанавливаются через аналитическое решение, что обеспечивает согласованность постановки и позволяет сосредоточиться на анализе внутренней аппроксимации.

Набор явных методов включает схему против потока, Лакса–Фридрихса, Лакса–Вендроффа, Мак-Кормака, варианты с искусственной вязкостью, а также TVD- и MUSCL-алгоритмы. Это делает приложение достаточно репрезентативным с методической точки зрения. Пользователь может сравнить простейший апвиндный метод, более точные схемы второго порядка и модификации, ориентированные на подавление осцилляций. Пример на рисунке (5.1).

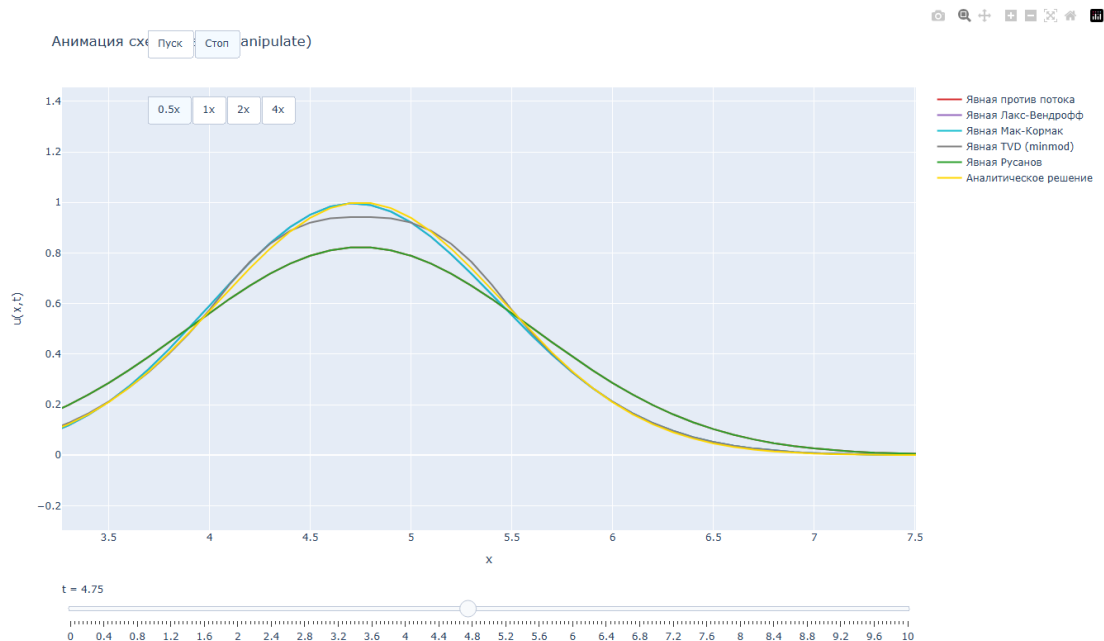


Рисунок 5.1

В реализации TVD- и MUSCL-подходов используются ограничители minmod и MC (рисунок (5.2)). Для каждого узла вычисляются локальные отношения соседних разностей, после чего строится корректирующий член. Та-

кой код несколько сложнее простых явных схем, но его включение в проект оправдано: без него приложение не давало бы возможности обсуждать современные монотонные подходы к численному решению гиперболических задач.

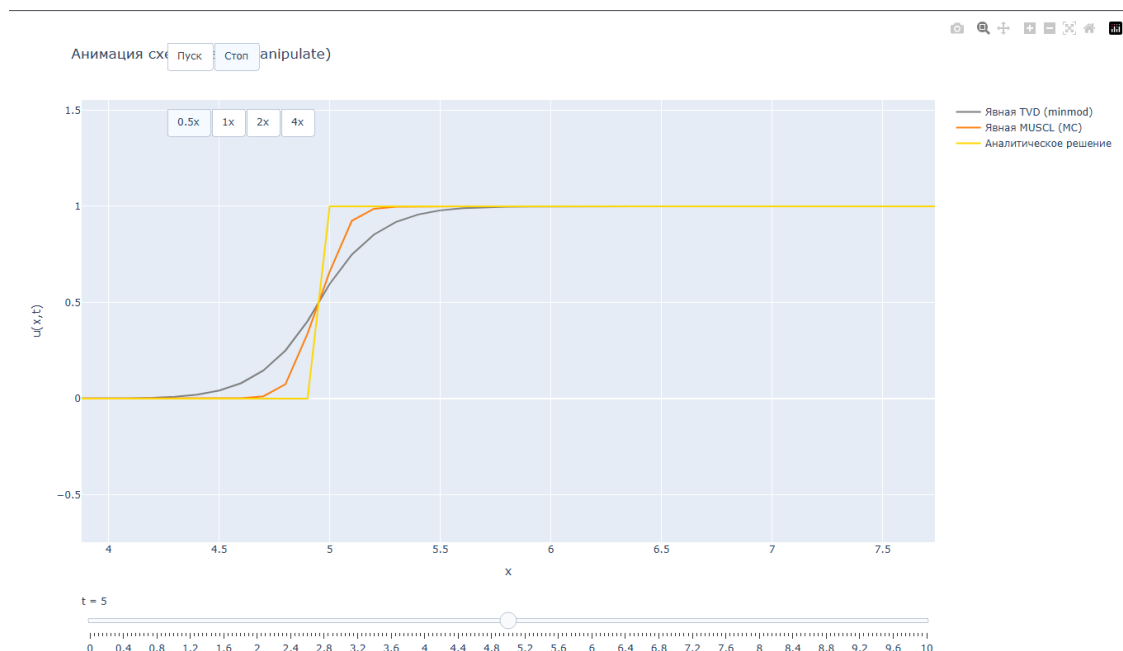


Рисунок 5.2

5.3 Реализация неявных схем

Неявные схемы разделены на два класса. Первый класс состоит из стандартных схем, для которых можно заранее задать структуру трёхдиагональной системы и решить её методом прогонки. Реализация метода прогонки выделена в отдельную функцию, принимающую поддиагональ, диагональ, наддиагональ и правую часть. Такая декомпозиция делает код повторно используемым и упрощает сопровождение.

Второй класс связан с пользовательскими матричными схемами. В этом случае из пользовательского описания извлекается набор диагоналей ленточной матрицы и формула правой части, после чего система собирается на каж-

дом временном шаге и передаётся решателю `solve_banded` из SciPy. Поддержка ленточной матрицы особенно полезна, поскольку многие практические схемы имеют именно такую структуру, а её хранение в плотном виде было бы избыточным.

Программно важно, что и стандартные, и пользовательские неявные схемы возвращают результат в одном и том же формате. Как на рисунке (5.3), здесь схема CN Central является пользовательской.

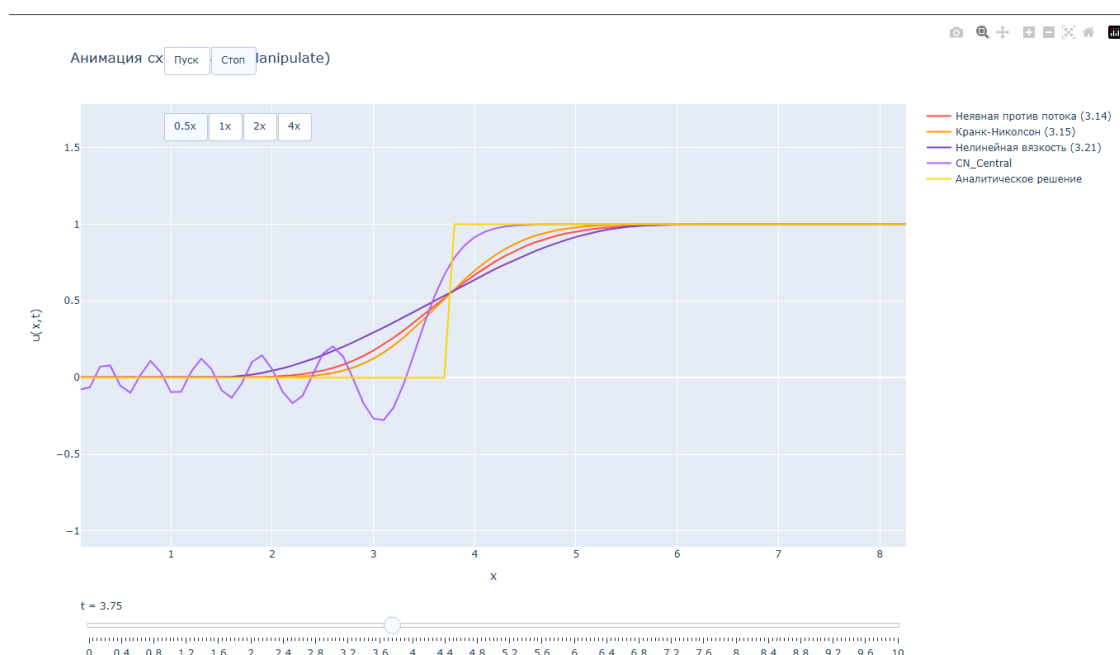


Рисунок 5.3

Благодаря этому весь остальной интерфейс — графики, таблицы ошибок, анализ сходимости — не зависит от того, каким способом было получено решение. Унифицированный формат результата является одним из главных факторов простоты расширения системы.

5.4 Расчёт ошибок и анализ сходимости

Функция расчёта норм принимает численное и аналитическое решения, а также пространственный шаг. Для каждого временного слоя вычисляются нормы L_1 , L_2 и L_∞ , показанных на рисунке (5.4)

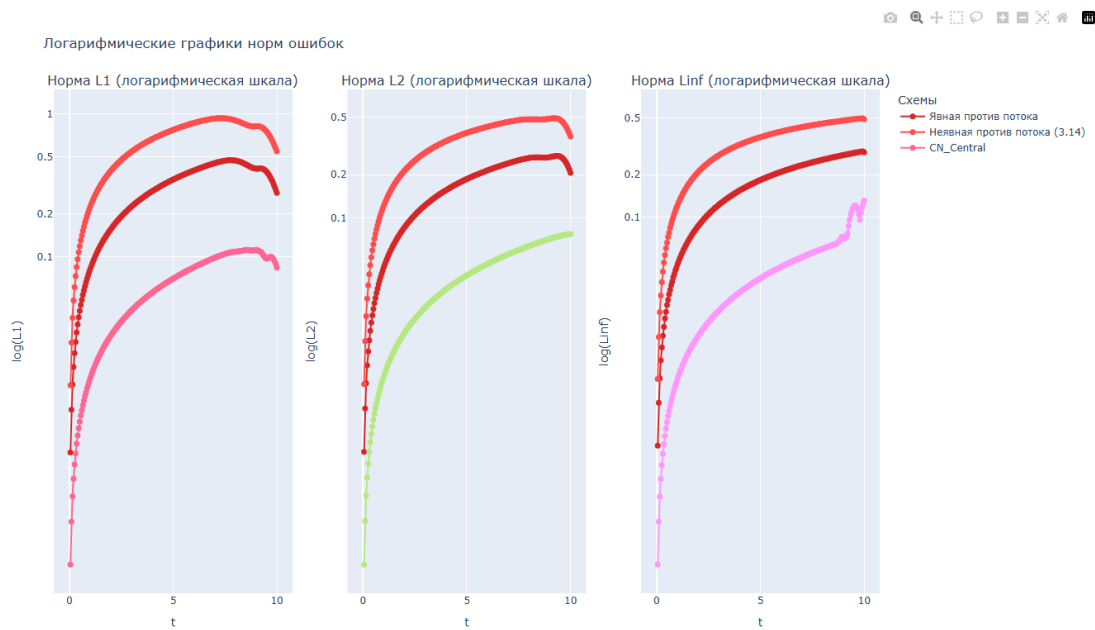


Рисунок 5.4

Далее результаты используются в нескольких сценариях:

- построение графиков динамики норм во времени;
- формирование итоговой сравнительной таблицы;
- сортировка схем по величине ошибки на последнем шаге;
- отображение текстового отчёта в интерфейсе.

Анализ сходимости реализован как серия расчётов одной выбранной схемы на последовательности сеток. Пользователь выбирает ровно один метод, после чего программа несколько раз строит решение при разных значениях N_x , вычисляет итоговую ошибку в норме L_2 и формирует логарифмический график зависимости ошибки от шага h . Такая процедура позволяет экспериментально оценить порядок сходимости и проверить, согласуется ли он с теоретическими ожиданиями. Пример на рисунке (5.5)

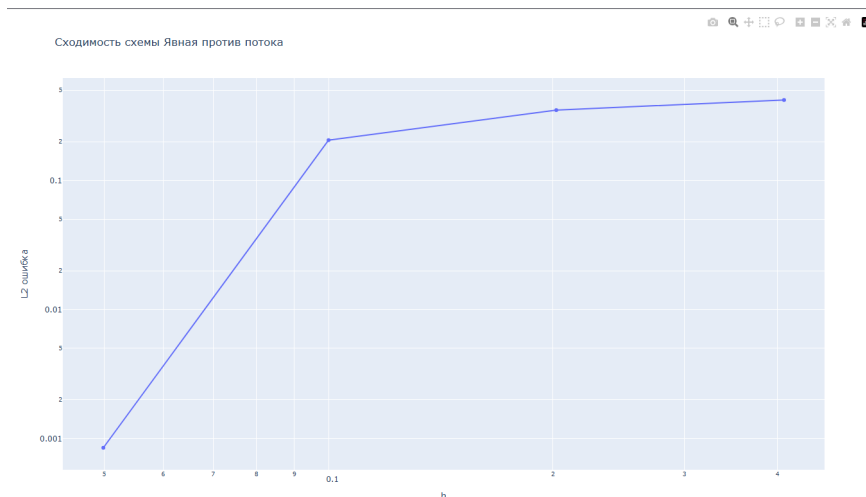


Рисунок 5.5

С точки зрения инженерии важно, что реализация анализа сходимости использует уже существующие функции контекста и схем. Это означает, что соответствующая функциональность появилась не за счёт дублирования кода, а благодаря повторному использованию базовых модулей. Подобный результат подтверждает правильность выбранной архитектуры.

5.5 Визуализация и экспорт

В приложении реализовано несколько типов визуализации. Анимация сравнения схем показывает эволюцию профиля во времени и содержит кнопки запуска, остановки и изменения скорости воспроизведения. Режим среза позволяет сравнить решения в фиксированный момент времени. Дополнительно доступны контурный график и трёхмерная поверхность аналитического решения, а также отображение характеристик.

Наличие нескольких представлений одной и той же информации имеет не декоративный, а методический смысл. Линейный график позволяет сопоставить формы профилей, контурное изображение показывает общую картину переноса, а трёхмерная поверхность подчёркивает связь между пространством, временем и значением решения. Визуализация характеристик особен-

но полезна при обсуждении физического смысла задачи и происхождения аналитической формулы.

Экспорт анимации реализован через отдельный рабочий объект, помещаемый в поток `QThread`. Это решение предотвращает блокировку пользовательского интерфейса при длительном формировании кадров. Пользователь видит окно прогресса, может отменить операцию и получает текстовый отчёт о результате. Подобная организация долгих операций является хорошей практикой разработки настольных приложений.

ГЛАВА 6 РЕЗУЛЬТАТЫ ВЫЧИСЛИТЕЛЬНЫХ ЭКСПЕРИМЕНТОВ

6.1 Организация экспериментов

Для проверки работоспособности и полезности приложения были выполнены серии вычислительных экспериментов. В качестве базового начального условия часто использовалась функция

$$\varphi_0(x) = \exp(-x^2), \quad (6.1)$$

поскольку она одновременно достаточно гладкая, хорошо визуализируется и позволяет наблюдать как чистое смещение профиля, так и эффекты численного размывания или осцилляций. В отдельных экспериментах рассматривались и более сложные пользовательские выражения, в том числе с использованием ступенчатых функций:

$$\varphi_0(x) = \theta(x), \quad (6.2)$$

Для пространственно-временной области использовались диапазоны, позволяющие видеть динамику решения на интервале, где профиль не успевает полностью выйти за пределы области наблюдения. Плотность сетки варьировалась от умеренной до повышенной, чтобы проследить влияние шагов h и τ на ошибку и устойчивость. Отдельно оценивался эффект изменения числа узлов при фиксированной постановке задачи для анализа сходимости.

6.2 Сравнение явных схем

Наиболее показательные различия между методами проявляются при сравнении явной схемы против потока, схем Лакса – Фридрихса и Лакса –

Вендроффа(рисунок (6.1)).

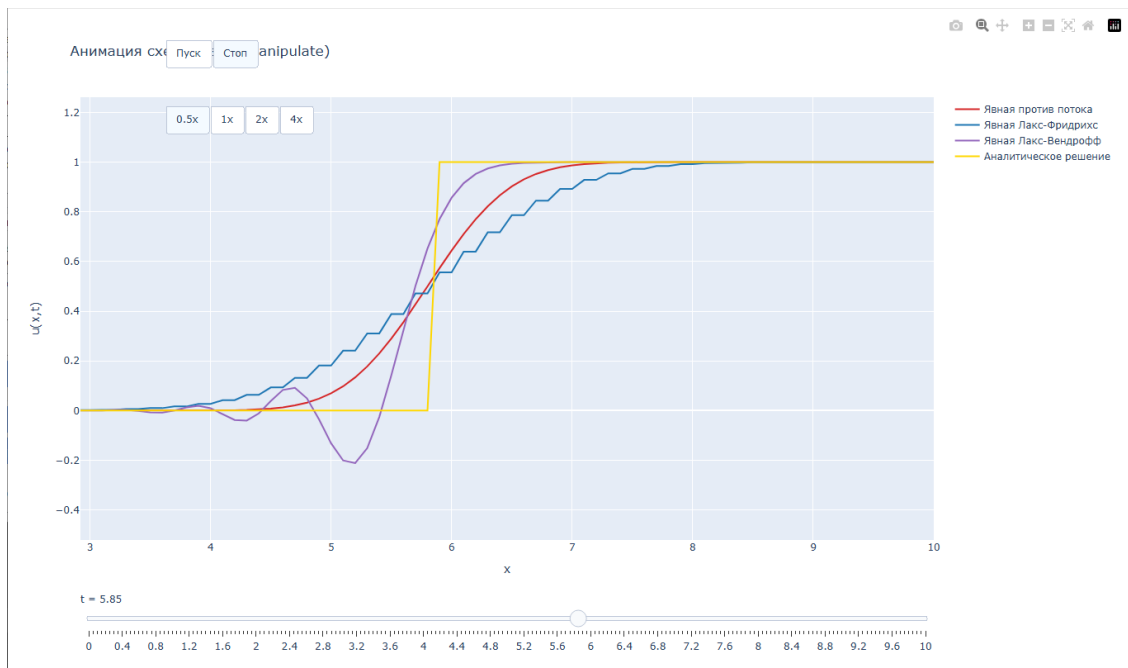


Рисунок 6.1

Апвиндный метод, как и ожидалось, демонстрирует устойчивое, но более размытое решение. При длительном переносе гладкий максимум профиля постепенно уплощается, а фронты сглаживаются. Этот эффект хорошо соответствует представлению о численной вязкости.

Схема Лакса–Фридрихса также проявляет заметную диссипацию. В ряде сценариев она оказывается даже более сглаживающей, чем апвиндная схема, хотя и остаётся полезной как простой эталонный метод. Схема Лакса–Вендроффа, напротив, лучше сохраняет форму гладкого профиля, но при недостаточно аккуратном выборе параметров может порождать волнообразные артефакты. Именно поэтому включение в приложение сразу нескольких схем оказывается методически оправданным: пользователь наблюдает не абстрактные рассуждения о порядке точности, а конкретные различия на одном и том же графике.

Схема Мак-Кормака занимает промежуточное положение и демонстрирует хороший баланс между точностью и наглядностью. В учебном контексте

она удобна ещё и тем, что двухшаговая структура предиктор–корректор легко объясняется и визуально связывается с получаемым результатом.

6.3 Роль ограничителей и схем TVD/MUSCL

При использовании ограничителей `minmod` и `MC` удалось продемонстрировать важный для гиперболических задач эффект: уменьшение паразитических колебаний без катастрофического роста диссипации. На гладких профилях TVD- и MUSCL-схемы дают более аккуратное решение по сравнению с простыми центральными подходами((5.2)), а на крутых фронтах ведут себя устойчивее.

Визуально различие между ограничителями можно описать так: `minmod` обеспечивает более строгую монотонность, но в ряде случаев сильнее сглаживает локальные особенности, тогда как `MC` допускает более точное восстановление наклона и лучше сохраняет форму, если решение остаётся достаточно гладким. Возможность сравнить эти варианты в едином интерфейсе полезна не только для обучения, но и для подготовки к исследовательской части выпускной квалификационной работы.

6.4 Сравнение неявных схем

Неявные методы (рисунок (5.3)) оказались особенно интересны в ситуациях, где явные схемы чувствительны к размеру временного шага. При увеличении τ устойчивость неявных схем сохраняется лучше, что подтверждает теоретические ожидания. При этом наблюдаются отличия по диссипативным свойствам: одни схемы сохраняют профиль сравнительно хорошо, другие дают более сглаженный результат, но при этом остаются стабильными.

Практически значимым оказался режим сравнения неявной центральной схемы, варианта Кранка–Николсона и методов с искусственной вязкостью. Кранк–Николсон демонстрирует высокий интерес как компромиссный

подход: он даёт хорошую точность, но при этом требует решения системы на каждом временном шаге. Схемы с регуляризацией и дополнительной вязкостью полезны для исследования влияния стабилизирующих членов на итоговую ошибку.

6.5 Пользовательские схемы как инструмент исследования

Особый класс экспериментов связан с пользовательскими схемами. В конфигурационном файле приложения были подготовлены примеры пользовательской явной схемы, центральной формулы и нескольких матричных модификаций по типу Кранка–Николсона. Эти примеры подтвердили работоспособность формата хранения и механизма загрузки: приложение корректно интерпретирует пользовательское описание, включает схему в общий список и позволяет анализировать её наравне со встроенными методами.

С методической точки зрения это одна из наиболее сильных сторон проекта. Пользователь может не только выбрать готовую схему, но и воспроизвести литературный метод по формуле из учебника или статьи. Если схема оказывается неустойчивой либо выдаёт большую ошибку, это тоже полезный результат, поскольку он непосредственно демонстрирует последствия неудачного выбора аппроксимации.

6.6 Анализ ошибок и таблицы норм

Расчёт норм ошибок показал, что итоговая оценка метода зависит от выбранной метрики. Для гладких начальных данных схемы второго порядка закономерно демонстрируют меньшие значения L_2 и L_1 при достаточном разрешении сетки. Однако в норме L_∞ на отдельных сценариях более строгие методы с ограничителями выглядят предпочтительнее из-за меньшей амплитуды локальных выбросов.

Наличие таблицы итоговых ошибок в интерфейсе оказалось очень полезным. Когда пользователь сравнивает сразу несколько схем, график даёт качественное представление, а таблица — количественное. Совместное использование этих форм представления результатов является важным достоинством разработанного приложения.

6.7 Анализ сходимости

При последовательном уточнении сетки приложение строит логарифмический график зависимости ошибки в норме L_2 от шага h . Такой график позволяет визуально оценить скорость убывания ошибки и, следовательно, экспериментально судить о порядке сходимости. Хотя в учебных условиях на конечных сетках нельзя ожидать идеального совпадения с асимптотической теорией, получаемые кривые демонстрируют качественно правильное поведение: схемы более высокого порядка показывают более быстрое снижение ошибки в зоне, где доминирует именно аппроксимационная составляющая.

Важным наблюдением является то, что порядок сходимости может искажаться при слишком грубых сетках, слишком большом числе Куранта или недостаточно гладком начальном условии. Следовательно, анализ сходимости в приложении полезен не только как подтверждение теории, но и как инструмент обсуждения ограничений самой теории при практическом вычислительном эксперименте.

6.8 Интерпретация результатов с точки зрения учебного процесса

Проведённые эксперименты показывают, что приложение действительно выполняет функцию интерактивной учебной лаборатории. Оно позволяет переходить от абстрактных формул к непосредственному наблюдению поведения схем, а также быстро изменять параметры и получать новый результат.

Для преподавателя это средство демонстрации, для студента — инструмент самостоятельного эксперимента, а для разработчика выпускной квалификационной работы — удобная база для последующих расширений.

Именно сочетание количественного анализа, визуализации и пользовательской расширяемости делает результат практики содержательно завершённым. Без визуализации продукт был бы просто вычислительным скриптом, без анализа ошибок — красивой, но слабо полезной оболочкой, а без поддержки пользовательских схем — закрытым демонстратором. Совместная реализация всех трёх составляющих обеспечивает целостность разработанной системы.

ГЛАВА 7 ТЕСТИРОВАНИЕ, ОТЛАДКА И ОЦЕНКА КАЧЕСТВА

7.1 Стратегия тестирования

Тестирование приложения проводилось на нескольких уровнях. На уровне входных данных проверялась корректность обработки числовых параметров, выражений для коэффициента переноса и начального условия, а также ситуаций с недопустимыми диапазонами по пространству и времени. На уровне вычислительных функций оценивалась способность схем возвращать массивы согласованных размеров, корректно обрабатывать границы и выдавать осмысленные ошибки при нарушении условий применимости.

На уровне сценариев пользовательского интерфейса проверялись основные действия: выбор режимов, расчёт анимации, построение срезов, вычисление норм, анализ сходимости, добавление и удаление пользовательских схем, экспорт результатов. Особое внимание уделялось сценариям, где потенциально возможна потеря управления интерфейсом, например при длительном экспорте анимации. Для таких случаев был реализован отдельный поток и окно прогресса.

7.2 Тестовые сценарии

Тест	Описание	Ожидаемый результат
T1	Ввод некорректного числа в поле границы сетки	Появляется сообщение об ошибке, программа не завершается аварийно.
T2	Задание $x_{\max} \leq x_{\min}$	Пользователь получает уведомление о некорректной области.

T3	Выбор аналитического режима для $a = a(x)$ с нулями коэффициента на сетке	Система сообщает, что коэффициент недопустим для такого режима.
T4	Построение анимации для набора встроенных схем	Открывается интерактивная анимация с ползунком времени.
T5	Расчёт норм ошибок для нескольких схем	Строятся графики норм и открывается таблица итоговых значений.
T6	Анализ сходимости без выбора схемы или при выборе нескольких схем	Отображается сообщение о необходимости выбрать ровно одну схему.
T7	Добавление пользовательской явной схемы	Схема появляется в списке и может участвовать в расчётах.
T8	Экспорт анимации в GIF или MP4	Создаётся файл результата, интерфейс остаётся отзывчивым.
T9	Отмена экспорта пользователем	Операция прекращается без зависания и без некорректных остаточных файлов.

Указанные сценарии подтвердили, что система в целом устойчива к типовым ошибкам пользователя и обеспечивает воспроизводимую работу в основном наборе задач. Разумеется, проект остаётся расширяемым, и его можно дополнительно покрыть автоматическими модульными тестами, однако для задач преддипломной практики полученный уровень надёжности можно считать достаточным.

7.3 Анализ производительности

Основная вычислительная нагрузка в приложении связана с временными циклами по сетке и построением анимаций. Для умеренных размеров сетки производительность Python с использованием NumPy оказывается достаточной. Явные схемы работают быстро, а неявные методы демонстрируют приемлемую скорость даже при необходимости решать линейную систему на каждом временном шаге, поскольку используются специализированные методы для ленточных матриц.

Более тяжёлой операцией является экспорт анимации. Здесь значительную роль играют не столько сами вычисления решения, сколько отрисовка кадров и кодирование итогового файла. Именно поэтому было принято решение выполнять экспорт в отдельном потоке. С точки зрения пользовательского опыта такое решение является критически важным: даже если операция занимает заметное время, пользователь сохраняет контроль над программой и может её отменить.

7.4 Ограничения текущей версии

Несмотря на достигнутый результат, текущая версия системы имеет ряд ограничений. Во-первых, приложение ориентировано на одномерное уравнение переноса и не рассматривает двумерные задачи. Во-вторых, для некоторых аналитических режимов требуется выполнение условий монотонности и ненулевого коэффициента. В-третьих, пользовательские схемы интерпретируются через выражения, что требует аккуратности при валидации и ограничивает набор поддерживаемых конструкций безопасным вычислительным контуром. Кроме того, проект в основном ориентирован на интерактивное исследование, а не на пакетные вычисления. Если в будущем потребуется запуск больших серий экспериментов без графического интерфейса, будет целесообразно выделить вычислительное ядро в отдельный модуль с командной строкой или API.

ГЛАВА 8 ПРАКТИЧЕСКИЕ РЕЗУЛЬТАТЫ И СФОРМИРОВАННЫЕ КОМПЕТЕНЦИИ

Преддипломная практика позволила получить не только программный продукт, но и набор профессиональных результатов, непосредственно связанных с подготовкой выпускной квалификационной работы. К основным практическим результатам можно отнести:

- разработку настольного приложения, решающего прикладную задачу исследования уравнения переноса;
- реализацию набора явных и неявных численных схем в единой архитектуре;
- внедрение аналитического эталона для ряда модельных случаев;
- создание механизма добавления пользовательских схем без изменения исходного кода;
- реализацию средств анализа ошибок, сходимости и устойчивости;
- подготовку отчётной документации, пригодной для дальнейшего использования в дипломном проекте.

С точки зрения профессиональных компетенций практика способствовала развитию способности:

1. формализовать прикладную задачу в виде математической модели;
2. выбирать адекватные численные методы и обосновывать их свойства;
3. проектировать структуру программной системы с учётом расширяемости;

4. реализовывать графические интерфейсы для научных приложений;
5. проводить вычислительный эксперимент и интерпретировать его результаты;
6. оформлять результаты работы в виде технической и отчётной документации.

Важным итогом стало понимание того, что качественный научно-образовательный программный продукт определяется не только корректностью алгоритмов, но и тем, насколько удобно пользователь может провести полный цикл исследования: ввести данные, получить результат, сравнить методы, зафиксировать выводы и воспроизвести эксперимент. Именно в таком комплексном виде задача была решена в рамках практики.

ГЛАВА 9 ПЕРСПЕКТИВЫ РАЗВИТИЯ ПРОЕКТА

Разработанное приложение имеет хорошие перспективы расширения. Прежде всего возможно развитие математической части. В проект можно добавить задачи с источником членом, нелинейные уравнения первого порядка, системы гиперболических уравнений, двумерные постановки и методы конечных объёмов. Это сделает платформу более универсальной и приблизит её к задачам реального инженерного моделирования.

С точки зрения программной архитектуры перспективным направлением является выделение вычислительного ядра в самостоятельный пакет с модульными тестами и документацией API. Тогда графический интерфейс станет лишь одним из способов взаимодействия с вычислительным модулем, а сами расчёты можно будет использовать в командной строке, в Jupyter-ноутбуках или в веб-приложении.

Полезным развитием могло бы стать внедрение системы сохранения и загрузки полных сценариев эксперимента, включающих не только пользовательские схемы, но и все параметры сетки, тип коэффициента, набор выбранных методов и параметры визуализации. Это ещё сильнее повысит воспроизводимость учебных и исследовательских работ.

Наконец, в перспективе можно добавить автоматизированную оценку порядка сходимости, экспорт таблиц ошибок в CSV или Excel, формирование отчётов в PDF по готовому шаблону и более строгую систему валидации пользовательских формул. Все эти направления логично продолжают уже реализованную архитектуру и могут лечь в основу дальнейшей выпускной квалификационной работы.

ЗАКЛЮЧЕНИЕ

В ходе преддипломной практики была решена комплексная задача разработки настольного приложения для исследования численных схем решения одномерного уравнения переноса. Выполненная работа объединила математический анализ, алгоритмизацию, проектирование архитектуры, разработку графического интерфейса, визуализацию результатов и подготовку итоговой документации. На практике это означало переход от теоретической постановки задачи к созданию реального инструмента, пригодного для учебно-исследовательского использования.

Созданное приложение обеспечивает построение аналитического решения для ряда модельных случаев, поддерживает широкий набор встроенных явных и неявных схем, позволяет пользователю добавлять собственные методы, вычисляет нормы ошибок, оценивает число CFL, строит графики сходимости и экспортирует анимации. Существенным достоинством проекта является то, что все перечисленные возможности объединены в одном интерфейсе и работают в рамках согласованной архитектуры.

Полученный результат имеет как практическую, так и методическую ценность. С одной стороны, разработанная система может служить лабораторной платформой для изучения вычислительных методов. С другой стороны, проект формирует прочную основу для дальнейшей дипломной разработки, в том числе для расширения постановки задачи и углубления математической части. Следовательно, цель преддипломной практики достигнута, а поставленные задачи решены в полном объёме.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Самарский А. А. Теория разностных схем. М.: Наука, 1989.
2. Годунов С. К., Рябенький В. С. Разностные схемы. М.: Наука, 1977.
3. LeVeque R. J. Numerical Methods for Conservation Laws. Basel: Birkhauser, 1992.
4. LeVeque R. J. Finite Volume Methods for Hyperbolic Problems. Cambridge: Cambridge University Press, 2002.
5. Richtmyer R. D., Morton K. W. Difference Methods for Initial-Value Problems. New York: Interscience Publishers, 1967.
6. Танненбаум Э., Поллард Г. Обыкновенные дифференциальные уравнения. М.: Мир, 1988.
7. NumPy Developers. NumPy Documentation. URL: <https://numpy.org/doc/>.
8. SciPy Developers. SciPy Documentation. URL: <https://docs.scipy.org/doc/scipy/>.
9. SymPy Development Team. SymPy Documentation. URL: <https://docs.sympy.org/>.
10. Riverbank Computing. PyQt5 Reference Guide. URL: <https://www.riverbankcomputing.com/static/Docs/PyQt5/>.
11. Qt Company. Qt Documentation. URL: <https://doc.qt.io/>.
12. Plotly Technologies Inc. Plotly Python Documentation. URL: <https://plotly.com/python/>.

13. Sweby P. K. High Resolution Schemes Using Flux Limiters for Hyperbolic Conservation Laws // SIAM Journal on Numerical Analysis. 1984. Vol. 21. No. 5. P. 995–1011.
14. Toro E. F. Riemann Solvers and Numerical Methods for Fluid Dynamics. Berlin: Springer, 2009.

ПРИЛОЖЕНИЕ А РУКОВОДСТВО ПОЛЬЗОВАТЕЛЯ

Данное приложение ориентировано на интерактивную работу, поэтому ниже приведён рекомендуемый порядок использования системы.

А.1 Запуск программы

После установки зависимостей приложение запускается как обычный Python-скрипт. Пользователь открывает главное окно, в котором доступны все основные режимы работы. При первом запуске список пользовательских схем загружается из конфигурационного JSON-файла, если такой файл уже был создан ранее.

А.2 Базовый сценарий работы

1. Выбрать тип коэффициента переноса в выпадающем списке.
2. При необходимости ввести аналитическую формулу коэффициента или параметры каталожного случая.
3. Ввести начальное условие $\varphi_0(x)$.
4. Задать границы и плотность пространственно-временной сетки.
5. Отметить интересующие встроенные и пользовательские схемы.
6. При необходимости включить отображение аналитического решения.
7. Нажать кнопку нужного режима анализа: анимация, срез, нормы ошибок, итоговые ошибки, сходимость и т.д.

А.3 Добавление пользовательской схемы

Для добавления новой схемы необходимо открыть диалоговое окно по кнопке добавления пользовательской схемы. Затем требуется:

1. ввести имя схемы;
2. выбрать тип реализации;
3. задать формулу явного перехода либо параметры неявной матрицы;
4. сохранить схему;
5. отметить её флажком в основном окне для участия в сравнении.

Пользовательские схемы сохраняются в JSON-файл и доступны при следующих запусках программы. Это особенно удобно при подготовке лабораторных работ и дипломных материалов, когда один и тот же набор методов используется многократно.

А.4 Интерпретация результатов

При анализе результатов рекомендуется учитывать несколько факторов одновременно:

- форму графика и наличие видимых осцилляций;
- значение норм ошибок и их динамику во времени;
- величину CFL-числа;
- влияние сгущения сетки на итоговую точность;
- различия между гладкими и негладкими начальными условиями.

Именно комплексный подход позволяет сделать корректный вывод о качестве численной схемы. Одна только визуализация или одна только таблица ошибок дают неполную картину.

ПРИЛОЖЕНИЕ В КАТАЛОГ РЕАЛИЗОВАННЫХ СХЕМ

В.1 Встроенные явные схемы

Схема	Класс метода	Краткая характеристика
Явная против потока	Апвиндная	Проста, устойчива при допустимом CFL, обладает численной вязкостью.
Явная Лакс–Фридрихс	Центральная усреднением	с Надёжна, но заметно сглаживает решение.
Явная Лакс–Вендрофф	Второй порядок	Лучше сохраняет гладкие профили, может осциллировать.
Явная вязкость	Стабилизированная	Добавляет искусственную вязкость, подавляет колебания.
Явная Мак-Кормак	Предиктор–корректор	Точна на гладких решениях, наглядна методически.
Явная TVD (minmod)	TVD	Подавляет осцилляции, более диссипативна.
Явная MUSCL (MC)	TVD/MUSCL	Сохраняет форму лучше minmod, требует аккуратной интерпретации.

Явная Русанов	Потоковая	Использует апвиндный поток, устойчива и проста.
Явная Годунов	Потоковая	Близка к апвиндной логике для данного класса задач.

В.2 Встроенные неявные схемы

Схема	Тип матрицы	Назначение
Неявная против потока (3.14)	Трёхдиагональная	Базовый неявный апвиндный метод.
Неявная центральная gen	Трёхдиагональная	Центральная аппроксимация для исследования устойчивости.
Неявная Лакс–Вендрофф (3.19)	Трёхдиагональная	Неявный аналог схемы повышенного порядка.
Кранк–Николсон (3.15)	Трёхдиагональная	Компромисс между точностью и устойчивостью.
Неявная апвинд (3.17)	Трёхдиагональная	Направленная схема с учётом знака скорости.
Линейная вязкость (3.20)	Трёхдиагональная	Стабилизация за счёт вязкостного члена.
Вязкость 4-го порядка (3.18)	Пентадиагональная	Более сложная регуляризация.

Нелинейная вязкость (3.21)	Трёхдиагональная	Коэффициент вязкости зависит от локального решения.
Регуляриз. Вендрофф (3.19)	Лакс– Трёхдиагональная	Комбинирует точность и стабилизацию.

ПРИЛОЖЕНИЕ С ФРАГМЕНТЫ КОНФИГУРАЦИИ И КОДА

В приложении пользовательские схемы сохраняются в JSON-представлении. Ниже приведён иллюстративный пример такого описания. Для совместимости с pdfL^AT_EX в листинге использованы ASCII-метки типов; в рабочем файле проекта допускаются и русскоязычные обозначения.

Listing C.1: Пример пользовательской конфигурации схем

```
1 {
2   "upwind_custom": [
3     "u - a * dt / dx * (u - u_left)",
4     "explicit"
5   ],
6   "Matrix_Crank_Nicolson": [
7     {
8       "diagonals": {
9         "0": "1",
10        "-1": "-a * dt / (2 * dx)",
11        "1": "a * dt / (2 * dx)"
12      },
13      "rhs": "u - a * dt / (2 * dx) * (u_right - u_left)"
14    },
15    "implicit_matrix"
16  ]
17 }
```

Такая форма хранения делает расширение приложения прозрачным для пользователя. Система читает конфигурацию, нормализует описание схемы и подключает её к общему вычислительному контуру. При необходимости список может быть дополнен новыми записями через графический интерфейс.

Ещё один показательный фрагмент связан с вычислением числа CFL:

Listing C.2: Расчёт числа Куранта в приложении

```
1 dx, dt = dxdt(ctx)
2 xg, tg = np.meshgrid(ctx.x, ctx.t, indexing="xy")
3 a_max = np.max(np.abs(ctx.a(xg, tg)))
4 cfl = a_max * dt / dx
```

Несмотря на краткость, данный код играет важную роль в пользовательском сценарии: он связывает математическое требование устойчивости с конкретными параметрами сетки и режима коэффициента переноса.